# Celestra cheatsheet – v6.6.0 – https://github.com/Serrin/Celestra/

## Core API

```
constant(value);
identity(value);
noop();
T();
F();
```

```
asyncConstant(value);
asyncIdentity(value);
asyncNoop();
asyncT();
asyncF();
```

```
eq(value1, value2);
gt(value1, value2);
gte(value1, value2);
lt(value1, value2);
lte(value1, value2);
```

```
RandomBoolean();

randomUUIDv7(v4=false);

nanoid([size=21[,alphabet="A-Za-z0-9_-"]]);

timestampID([size=21[,alphabet="123456789ABCD
EFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz
"]]);
```

```
delay(milisec).then(callback);

bind(function,context);
unBind(function);
curry(function);
compose(function1[,functionN]);
pipe(function1[,functionN]);
once(function);
tap(function): function(value);
```

```
extend([deep,]target,source1[,sourceN]);
sizeIn(object);
pick(object,keys);
omit(object,keys);
assoc(object,key,value);
getUrlVars([str=location.search]);
obj2string(object);
```

```
BASE16;
BASE32;
BASE36;
BASE58;
BASE62;
WORDSAFEALPHABET;
```

```
createPolyfillMethod(object,property,value);

createPolyfillProperty(object,property,value);

assert(condition[,message|error]);

VERSION;
```

## DOM API

```
qsa(selector[,context]);
qs(selector[,context]);
domReady(callback);
domClear(element);
domCreate(type[,properties[,innerHTML]]);
domCreate(element descriptive object);
domToElement(htmlString);
domGetCSS(element[,property]);
domSetCSS(element,property,value);
domSetCSS(element,properties);
domFadeIn(element[,duration[,display]]);
domFadeOut(element[,duration]);
domFadeToggle(element[,duration[,display]]);
domShow(element[,display]);
domHide(element);
domToggle(element[,display]);
domIsHidden(element);
domScrollToTop();
domScrollToBottom();
domScrollToElement(element[,top=true]);
domSiblings(element);
domSiblingsPrev(element);
domSiblingsLeft(element);
domSiblingsNext(element);
domSiblingsRight(element);
domGetCSSVar(name);
domSetCSSVar(name,value);
importScript(script1[,scriptN]);
importStyle(style1[,styleN]);
setFullscreenOn(selector);
setFullscreenOn(element);
setFullscreenOff();
getFullscreen();
form2array(form);
form2string(form);
getDoNotTrack();
getLocation(success[,error]);
createFile(filename,content[,dType]);
```

| Polyfills | String API | Math API | |
|---|---|---|---|
| crypto.randomUUID();<br><br>Error.isError();<br><br>globalThis;<br><br>Math.sumPrecise();<br><br>globalThis.AsyncFunction();<br><br>globalthis.AsyncGeneratorFunction();<br><br>globalThis.GeneratorFunction(); | b64Decode(string);<br>b64Encode(string);<br><br>strCodePoints(string);<br>strFromCodePoints(iterator);<br><br>strAt(string,index[,newChar]);<br><br>strCount(string, substring);<br><br>strSplice(str,index,count[,add]);<br><br>strTruncate(string);<br><br>strReverse(string);<br><br>strUpFirst(string);<br>strDownFirst(string);<br>strCapitalize(string);<br>strPropercase(string);<br>strTitlecase(string);<br><br>strHTMLEscape(string);<br>strHTMLRemoveTags(string);<br>strHTMLUnEscape(string); | sum(value1[,valueN]);<br>avg(value1[,valueN]);<br>product(value1[,valN]);<br><br>clamp(value,min,max);<br>minmax(value,min,max);<br>inRange(value,min,max);<br><br>signbit(value);<br><br>randomInt([max]);<br>randomInt(min,max);<br><br>randomFloat([max]);<br>randomFloat(min,max);<br><br>add(value1,value2);<br>sub(value1,value2);<br>mul(value1,value2);<br>div(value1,value2);<br><br>divMod(value1,value2);<br>mod(value1,value2);<br><br>pow(base, power); | isEven(value);<br>isOdd(value);<br>isInt8(value);<br>isInt16(value);<br>isInt32(value);<br>isUInt32(value);<br>isUInt8(value);<br>isUInt16(value);<br>isBigInt64(value);<br>isBigUInt64(value);<br>isFloat16(value);<br>isFloat(value);<br><br>toInteger(value);<br>toIntegerOrInfinity(val);<br>toInt8(value);<br>toInt16(value);<br>toInt32(value);<br>toUInt8(value);<br>toUInt16(value);<br>toUInt32(value;<br>toBigInt64(value);<br>toBigUInt64(value);<br>toFloat16(value);<br>toFloat32(value); |

| Cookie API |
|---|
| getCookie([name]);<br>hasCookie(name);<br>setCookie(name,value[,hours=8760[,path="/"[,domain[,secure[,SameSite="Lax"[,HttpOnly]]]]]]);<br>setCookie(Options object: properties are the same as the parameters);<br>removeCookie(name[,path="/"[,domain[,secure[,SameSite="Lax"[,HttpOnly]]]]]);<br>removeCookie(Options object: properties are the same as the parameters);<br>clearCookies([path="/"[,domain[,sec[,SameSite="Lax"[,HttpOnly]]]]]);<br>clearCookies(Options object: properties are the same as the parameters); |

| Collections API | | Type API |
|---|---|---|
| castArray(value);<br>arrayDeepClone(array);<br>arrayMerge(target,source1[,sourceN]);<br>arrayAdd(array,value);<br>arrayClear(array);<br>arrayRemove(array,value[,all = false]);<br>arrayRemoveBy(array,callback[,all=false]);<br><br>arrayRange([start=0[,end = 99[,step = 1]]]);<br>arrayCycle(iterator[,n = 100]);<br>arrayRepeat(value[,n = 100]);<br><br>iterRange([start=0[,step=1[,end=Infinity]]]);<br>iterCycle(iterator[,n = Infinity]);<br>iterRepeat(value[,n = Infinity]);<br><br>count(iterator,callback);<br><br>compact(iterator);<br><br>unique(iterator[,resolver]);<br><br>slice(iterator[,begin=0[,end = Infinity]]);<br><br>withOut(iterator,filterIterator);<br><br>reduce(iterator,callback[,initialvalue]);<br><br>take(iterator[,n = 1]);<br>takeWhile(iterator,callback);<br>takeRight(iterator[,n = 1]);<br>takeRightWhile(iterator,callback);<br><br>drop(iterator[,n = 1]);<br>dropWhile(iterator,callback);<br>dropRight(iterator[,n = 1]);<br>dropRightWhile(iterator,callback); | forEach(iterator,callback);<br>map(iterator,callback);<br>enumerate(iterator[,offset = 0]);<br>size(collection);<br><br>every(iterator,callback);<br>some(iterator,callback);<br>none(iterator,callback);<br><br>includes(collection,val[,comparator]);<br>find(iterator,callback);<br>findLast(iterator,callback);<br>filter(iterator,callback);<br>reject(iterator,callback);<br>partition(iterator,callback);<br><br>zip(iterator1[,iteratorN]);<br>unzip(iterator);<br>zipObj(iterator1,iterator2);<br>shuffle(iterator);<br><br>min(value1[,valueN]);<br>max(value1[,valueN]);<br>sort(iterator[,numbers = false]);<br>reverse(iterator);<br><br>item(iterator,index);<br>nth(iterator,index);<br>first(iterator);<br>head(iterator);<br>last(iterator);<br>initial(iterator);<br>tail(iterator);<br><br>flat(iterator);<br>concat(iterator1[,iteratorN]);<br>join(iterator[,separator = ","]); | typeOf(value);<br>is(val[,expectedType[,Throw=false]]);<br>isTypedCollection(iter,expectedType,<br>Throw=false);<br>isSameType(value1,value2);<br>isSameInstance(val1,val2,Contructor);<br>isDeepStrictEqual(value1,value2);<br>isCoercedObject(object);<br>isEmpty(value);<br>isNull(value);<br>isUndefined(value);<br>isNullish(value);<br>isNonNullable(value);<br>isNonNullablePrimitive(value);<br>isPlainObject(value);<br>isFunction(value);<br>isGeneratorFunction(value);<br>isAsyncFunction(value);<br>isAsyncGeneratorFunction(value);<br>isArrowFunction(value);<br>isProxy(value);<br>isElement(value);<br>isRegexp(value);<br>isArraylike(value);<br>isTypedArray(value);<br>isIterator(value);<br>isIterable(value);<br>isAsyncIterator(value);<br>isAsyncIterable(value);<br>isPropertyKey(value);<br>toPropertyKey(value);<br>isPrimitive(value);<br>toPrimitive(value);<br>isObject(value);<br>toObject(value);<br>isIndex(value); *and* toIndex(value);<br>isLength(value); *and* toLength(value);<br>toSafeString(value); |

| How to import | |
|---|---|
| **Celestra for browser: *celestra.browser.js*** | **Celestra for Node.js and Deno: *celestra.node.mjs*** |
| <br>**<script type="module">**<br>*// import the defaultExport object*<br>import defaultExport from "./celestra.browser.js";<br>globalThis.celestra = defaultExport;<br>globalThis.CEL = defaultExport;<br>**</script>**<br><br>**<script type="module">**<br>*// import with default with name*<br>import { default as celestra } from "./celestra.browser.js";<br>globalThis.celestra = celestra;<br>globalThis.CEL = celestra;<br>**</script>**<br><br>**<script type="module">**<br>*// import all into a new celestra object*<br>import * as celestra from "./celestra.browser.js";<br>globalThis.celestra = celestra;<br>globalThis.CEL = celestra;<br>**</script>**<br><br>**<script type="module">**<br>*// import some functions*<br>import { first, map } from "./celestra.browser.js";<br>globalThis.first = first;<br>globalThis.map = map;<br>**</script>**<br><br>**<script type="module">**<br>// dynamic import<br>const celestra = await import("./celestra.browser.js");<br>globalThis.celestra = celestra;<br>globalThis.CEL = celestra;<br>**</script>** | *// import the defaultExport object*<br><br>import defaultExport from "./celestra.node.mjs";<br>globalThis.celestra = defaultExport;<br>globalThis.CEL = defaultExport;<br><br>*// import with default with name*<br><br>import { default as celestra } from "./celestra.node.mjs";<br>globalThis.celestra = celestra;<br>globalThis.CEL = celestra;<br><br>*// import all into a new celestra object*<br><br>import * as celestra from "./celestra.node.mjs";<br>globalThis.celestra = celestra;<br>globalThis.CEL = celestra;<br><br>*// import some functions*<br><br>import { first, map } from "./celestra.node.mjs";<br>globalThis.first = first;<br>globalThis.map = map;<br><br>// dynamic import<br><br>const celestra = await import("./celestra.node.mjs");<br>globalThis.celestra = celestra;<br>globalThis.CEL = celestra; |
| | **Removed APIs in the *celestra.node.mjs*** |
| | DOM API \| Cookie API |

| Removed Polyfills – Available in celestra-polyfills.dev.js and celestra-polyfills.min.js | | |
|---|---|---|
| **v3.1.0** | **v3.8.0** | **v5.6.0** |
| Array.from();<br>Array.of();<br>Array.prototype.copyWithin();<br>Array.prototype.fill();<br>Array.prototype.find();<br>Array.prototype.findIndex();<br>Object.create();<br>String.fromCodePoint();<br>String.prototype.codePointAt();<br>String.prototype.endsWith();<br>String.prototype.startsWith();<br>Math.acosh();<br>Math.asinh();<br>Math.atanh();<br>Math.cbrt();<br>Math.clz32();<br>Math.cosh();<br>Math.expm1();<br>Math.fround();<br>Math.hypot();<br>Math.imul();<br>Math.log1p();<br>Math.log10();<br>Math.log2();<br>Math.sign();<br>Math.sinh();<br>Math.tanh();<br>Math.trunc();<br>Number.EPSILON;<br>Number.isNaN();<br>isNaN();<br>Number.isInteger();<br>Number.isFinite();<br>Number.isSafeInteger();<br>Number.parseInt();<br>Number.parseFloat(); | Array.prototype.values();<br>Array.prototype.includes();<br><br>ChildNode.after();<br>ChildNode.before();<br>ChildNode.remove();<br>ChildNode.replaceWith();<br><br>Element.prototype.closest();<br>Element.prototype.getAttributeNames();<br>Element.prototype.matches();<br>Element.prototype.toggleAttribute();<br><br>ParentNode.append();<br>ParentNode.prepend();<br><br>String.prototype[Symbol.iterator]();<br>String.prototype.includes();<br>String.prototype.repeat();<br><br>NodeList.prototype.forEach();<br><br>Object.assign();<br>Object.entries();<br>Object.getOwnPropertyDescriptors();<br>Object.values();<br><br>RegExp.prototype.flags;<br><br>window.screenLeft;<br>window.screenTop; | Array.prototype.at();<br>Array.prototype.findLast();<br>Array.prototype.findLastIndex();<br>Array.prototype.flat();<br>Array.prototype.flatMap();<br>Number.MIN_SAFE_INTEGER;<br>Number.MAX_SAFE_INTEGER;<br>Object.fromEntries();<br>Object.is();<br>String.prototype.at();<br>String.prototype.matchAll();<br>String.prototype.padStart();<br>String.prototype.padEnd();<br>String.prototype.replaceAll();<br>String.prototype.trimStart();<br>String.prototype.trimLeft();<br>String.prototype.trimEnd();<br>String.prototype.trimRight();<br>Typedarray.prototype.at();<br>TypedArray.prototype.findLast();<br>TypedArray.prototype.findLastIndex(); |
| | | **v5.9.0** |
| | | BigInt.prototype.toJSON(); |
| | | **v6.5.0** |
| | | Array.fromAsync();<br>Array.prototype.toReversed();<br>Array.prototype.toSorted();<br>Array.prototype.toSpliced();<br>Array.prototype.with();<br>Map.groupBy(); *and* Object.groupBy();<br>Object.hasOwn();<br>TypedArray.prototype.toReversed();<br>TypedArray.prototype.toSorted();<br>TypedArray.prototype.with(); |